NED UNIVERSITY OF ENGINEERING AND TECHNOLOGY

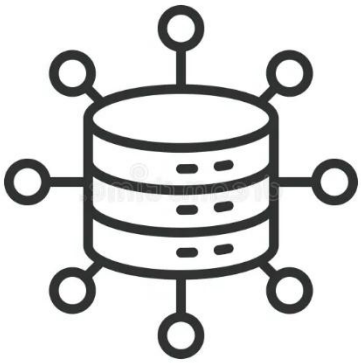
DBMS COMPLEX ENGINEERING PROJECT
Progressive Report: HappyDent Dental Management System

Course Code: CS-222

Submitted to:
Dr. Syed Zaffer Qasim

Submitted by:

Usman Rasheed Siddiqui	(CS-24038)
Huzaifa Hanif	(CS-24039)
Salman Raza	(CS-24043)



Clinic Page:
https://www.instagram.com/happydent.pk/

Table of Contents

Introduction: 4
Requirements: 4
Overview 4
For Patients 4
For Medical Staff (Doctors & Nurses) 4
For Administrators 5
Formulation and Analysis: 5
Gathering Information 5
Identifying Entities 5
Defining Attributes 6
Defining Relationships 6
Establishing Cardinalities and Participation 6
Entity-Relationship Model (ER Model) 8
Reduction of ER Model to Tables / Relational Schema 9
Strong Entity Sets: 9
Schema Derived From ISA Relationships: 9
Multivalued Attributes Relational Schema: 9
Week Entity Sets: 10
Schemas Derived From A Relationship Set Involving Strong Entity Sets: 10
Normalization 11
Staff: 11
Patient_Record 13
Record_Clinical_Notes 14
Medical 15
Doctor 16
Supplier 17
Bill 18
Administrative 19
Nurse 20
Staff_Phone 20
Patient_Phone 21
Doctor_Specialization 22
Doctor_Qualification 23
Salary 23
Products 24
Patients 26
Procurement 27
Manages 28

Appointment 28
Final Normalized Tables 30
For Strong Entity Sets 30
For Schema Derived From Isa Relationships 30
For Multivalued Attributes Relational Schema 30
For Weak Entity Sets 31
For Schemas Derived From A Relationship Set Involving Strong Entity Sets 31
DDL Statements For Creating Tables 32
Strong Entity Sets 32
ISA Relationships (Specialization) 34
3. Multivalued Attributes 35
Weak Entity Sets 35
Schemas Derived From A Relationship Set Involving Strong Entity Sets 36
DDL Statements For Indices: 36
Acknowledgement 38
Usman Rasheed Siddiqui (CS-24038) 38
Huzaifa Hanif (CS-24039) 38
Salman Raza (CS-24043) 38

Introduction:

Seeing a well-known dental clinic in the local area still relying on hardcopy records and manual processes highlights a major inefficiency in modern healthcare management. Maintaining patient files on paper, handling appointments manually, and tracking treatments without a proper system often leads to errors, data loss, and delays in service.

To address these issues, the **Happydent Dental Management System** is proposed as a database-driven solution to digitize and streamline clinic operations. The system is designed to manage patient records, staff information, appointments, treatments, and payments in an organized and efficient manner. Giving easy access to the patient to look around their medical or procedure history.

This project focuses on designing the database for the system by identifying key entities, defining their relationships, and transforming them into structured tables. Normalization techniques are applied to ensure data consistency and eliminate redundancy, followed by the implementation of SQL statements.

Overall, the Happydent system aims to improve efficiency, accuracy, and reliability in managing dental clinic operations through a well-structured database.

Requirements:

Overview

The database design for the **HappyDent Dental and Aesthetic Clinic** requires developing an efficient and normalized database model to store data about patients, clinical staff, medical records, and inventory. The system is designed to bridge the gap between clinical operations and administrative management, providing an all-in-one digital solution.

For Patients

The service will enable patients to:

- **Manage Profiles:** Store personal information, medical history (including allergies and chronic conditions like hypertension or diabetes), and contact details.
- **Appointment Scheduling:** View available slots and book appointments with specific doctors.
- **Financial Transparency:** Access bills generated from their visits and track payment history for various dental procedures.

For Medical Staff (Doctors & Nurses)

The system will allow medical professionals to:

- **Maintain Patient Records:** Create detailed patient records following each visit, including treatment plans, clinical notes, and prescriptions.
- **Track Availability:** Manage their specific specializations, qualifications, and daily availability for appointments.
- **Procedural History:** Record specific procedures performed to ensure accurate historical data and clinical continuity.

For Administrators

Administrative users will have the authority to:

- **Manage Staff & Payroll:** Oversee staff roles, clearance levels, and salary distributions (including bonuses and deductions).
- **Inventory & Procurement:** Manage suppliers and product stock (dental tools, medicine, and materials) to ensure the clinic is always fully equipped.
- **Facility Management:** Organize clinic departments and assign specific rooms to optimize patient flow.
- **Financial Oversight:** Monitor the billing process and audit the clinic's total revenue and budget authority.

Formulation and Analysis:

The thought process behind **HappyDent** ensures that clinical operations (treatments), administrative tasks (payroll/inventory), and patient management (billing/records) function as a unified, error-free system. The ER model serves as the blueprint for the entire database schema.

Gathering Information

For requirement gathering, multiple sources were considered to understand the real-world workflow of a dental clinic. A primary source was the observation of existing manual records maintained by the clinic. Photographs of individual patient records were taken, which provided insight into how patient data, treatments, and visit details are documented in practice. However, only a limited subset of this data has been included in the project due to privacy concerns and the complexity involved in handling large volumes of real-world data. Additionally, modern tools such as Large Language Models (LLMs) were utilized to convert the captured images into structured text format, which was then organized and entered into database tables. This approach helped in accurately transforming unstructured real-world data into a usable digital format for database design and analysis.

Identifying Entities

Based on the clinical requirements, the following primary entities were identified:

- **Staff:** The overarching entity for all employees.

- **Patient:** The individuals receiving care.
- **Department:** The various wings of the clinic (e.g., Orthodontics).
- **Patient_Record:** The medical and clinical history of a visit.
- **Bill:** The financial document generated post-treatment.
- **Supplier & Product:** The entities managing clinic inventory.
- **Salary:** The financial entity for staff compensation.

Defining Attributes

Attributes define the characteristics of each entity and establish Primary Keys (PK) for unique identification:

- **Staff:** Includes staff_id (PK), name (composite: first, last), email, DOB, gender, address, and phone (multi-valued).
- **Patient:** Includes patient_id (PK), name, email, DOB, gender, and specific medical flags like diabetes, hypertension, and allergies.
- **Department:** Consists of dept_id (PK), dept_name, and room_number.
- **Patient_Record:** Includes record_id (PK), visit_date, treatment_plan, clinical_notes, and prescription. amount_due is a derived attribute.
- **Bill:** Contains bill_id (PK), total_amount, and date_issued.
- **Salary:** A weak entity dependent on Staff, containing payment_date, amount, bonus, and deductions.
- **Product:** Includes item_id (PK), item_name, category, quantity, and unit_price.

Defining Relationships

Relationships depict the interconnection between clinical and business operations:

- **Works_In:** Links Staff and Department.
- **Works_With / ISA:** The Staff entity uses **Specialization** to branch into Administrative and Medical staff. Medical further specializes into Doctor and Nurse (**Disjoint Relationship**).
- **Has:** Links Patient to Patient_Record. A patient can have multiple historical records.
- **Creates:** A Doctor creates a Patient_Record during a visit.
- **Generates:** A Patient_Record generates a Bill.
- **Procurement:** Administrative staff manage the ordering of Products from Suppliers.
- **Manages:** Administrative staff oversee the inventory of Products.

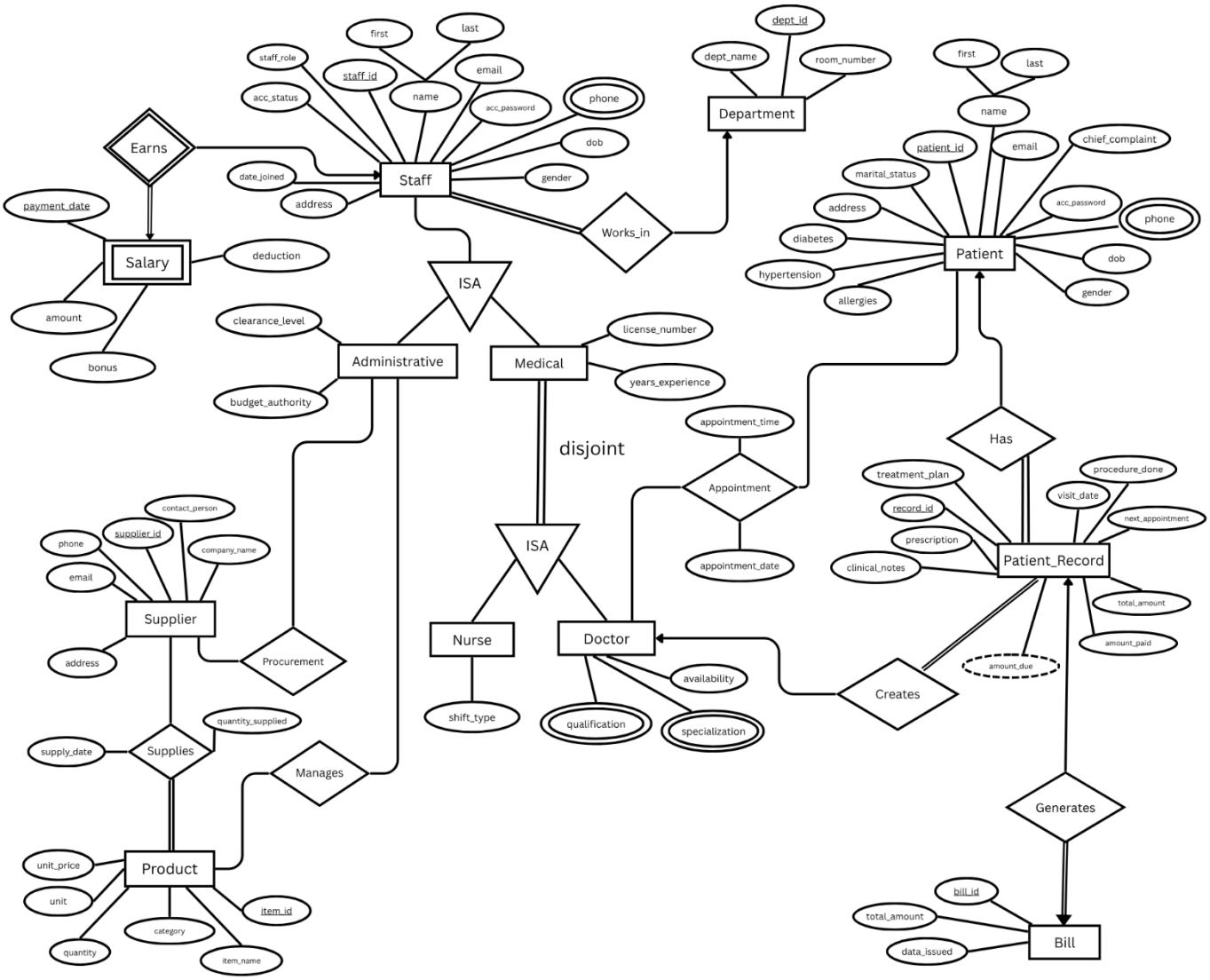
Establishing Cardinalities and Participation

Cardinality and participation define the business rules of the clinic:

- **Works_In (Many-to-One):** Many staff members work in one department. Participation is **Total** for Staff (every staff member must belong to a department).

- **Has (One-to-Many):** One patient can have many patient records, but a specific record belongs to only one patient. **Total Participation** on the Patient_Record i.e. Records can't exist without a patient.
- **Generates (One-to-One):** One patient record generates exactly one bill. Since a record cannot exist without a financial trail and vice versa, this is **Total Participation** for both.
- **Earns (One-to-Many):** One staff member earns many salary payments over time. **Salary** is a weak entity with total participation.
- **Creates (One-to-Many)**: One doctor can create many patient records. Patient_Record has total participation i.e records can't be formed without a doctor.
- **Appointment** (**Many-to-Many):** Many doctors can have many appointments with patients. (One Doctor can have slot only if empty else no)
- **Procurement** (**Many-to-Many):** Many administrators can deal with many suppliers.
- **Manages** (**Many-to-Many):** Many administrators can manage many products.
- **Supplies** (**Many-to-Many):** Many suppliers can supply many products. Product have total participation i.e. cannot exist without a definite supplier

Entity-Relationship Model (ER Model)



Reduction of ER Model to Tables / Relational Schema

Strong Entity Sets:

1. **Department**(**<u>dept_id</u>**, dept_name, room_number)

2. **Patient**(**<u>patient_id</u>**, name_first, name_last, email, chief_complaint, acc_password, dob, gender, address, marital_status, diabetes, hypertension, allergies)

3. **Supplier**(**<u>supplier_id</u>**, contact_person, company_name, email, phone, address)

4. **Product**(**<u>item_id</u>**, item_name, category, quantity, unit, unit_price,)
   Updated: Product(**<u>item_id</u>**, item_name, category, quantity, unit, unit_price, supply_date, quatity_supplied, *supplier_id*)

5. **Staff**(**<u>staff_id</u>**, name_first, name_last, email, acc_password, dob, gender, address, date_joined, acc_status, staff_role)
   Updated: Staff(**<u>staff_id</u>**, name_first, name_last, email, acc_password, dob, gender, address, date_joined, acc_status, staff_role, *dept_id*)

6. **Patient_Record**(**<u>record_id</u>**, treatment_plan, prescription, procedure_done, visit_date, next_appointment, total_amount, amount_paid, *patient_id*)
   Updated: Patient_Record(**<u>record_id</u>**, treatment_plan, prescription, clinical_notes, procedure_done, visit_date, next_appointment, total_amount, amount_paid, *patient_id*, *staff_id*)

7. **Bill** (**<u>bill_id</u>**, amount_paid, payment_date, payment_method)
   Updated: Bill (**<u>bill_id</u>**, amount_paid, payment_date, payment_method, *record_id*)

Schema Derived From ISA Relationships:

1. **Administrative**(**<u>staff_id</u>**, clearance_level, budget_authority)
2. **Medical**(**<u>staff_id</u>**, license_number, years_experience)
3. **Nurse**(**<u>staff_id</u>**, shift_type)
4. **Doctor**(**<u>staff_id</u>**, availability)

Multivalued Attributes Relational Schema:

1. **Staff_Phone**(**<u>staff_id</u>**, phone)
2. **Patient_Phone**(**<u>patient_id</u>**, phone)
3. **Doctor_Specialization**(**staff_id,** specialization)

4. **Doctor_ Qualification** (**staff_id,** qualification)
5. **Record_Clinical_Notes(record_id**, clincal_notes)

Week Entity Sets:

1. **Salary(staff_id**, **payment_date**, amount, bonus, deduction)

Schemas Derived From A Relationship Set Involving Strong Entity Sets:

1. **Works_in(staff_id,** dept_id)
   This relation has total participation towards Staff hence we will move it's Attributes to Staff Schema.
   Staff(staff_id, name_first, name_last, email, acc_password, dob, gender, address, date_joined, acc_status, staff_role, *dept_id*)
2. **Procurement(staff_id,** supplier_id)
3. **Supplies(item_id**, **supplier_id,** supply_date, quatity_supplied)
   This relation has total participation towards Product hence we will move it's Attributes to Product Schema.
   Product(item_id, item_name, category, quantity, unit, unit_price, *supplier_id*, supply_date, quatity_supplied)
4. **Manages(item_id**, **staff_id**)
5. **Appointment(staff_id**, **patient_id,** appointment_date, appointment_time)
6. **Has(record_id**, patient_id)
   This relation has total participation towards Patient_Record hence we will move it's Attributes to Patient_Record Schema.
   Patient_Record(record_id, treatment_plan, prescription, clinical_notes, procedure_done, visit_date, next_appointment, total_amount, amount_paid, *patient_id*)
7. **Creates(record_id**, staff_id)
   This relation has total participation towards Patient_Record hence we will move it's Attributes to Patient_Record Schema.
   Patient_Record(record_id, treatment_plan, prescription, clinical_notes, procedure_done, visit_date, next_appointment, total_amount, amount_paid, *patient_id*, *staff_id*)
8. **Generates(bill_id**, record_id)
   This relation has total participation towards Bill hence we will move it's Attributes to Bill Schema.
   Bill(bill_id, amount_paid, payment_date, payment_method, *record_id*)

Normalization

Staff:

staff_id	name_first	name_last	email	acc_password	dob	gender	address	date_joined	acc_status	staff_role	dept_id
ST-000001	Dr. Arsalan	Ahmed	arsalan.ahmed88@gmail.com	Arsalan@8812	12/04/1988	Male	House 42, Block L, North Nazimabad	2023-01-15	Active	Senior Dentist	D-101
ST-000002	Sarah	Munir	sarah_munir92@hotmail.com	Sarah@9205	25/08/1992	Female	Flat 302, Al-Azam Square, Gulshan	2024-05-10	Active	Dental Assistant	D-101
ST-000003	Muhammad	Bilal	m.bilal.khi@outlook.com	Bilal@9511	05/11/1995	Male	Sector 11-G, New Karachi	2025-02-20	Active	Manager	D-102
ST-000004	Dr. Hiba	Siddiqui	hiba.siddiqui90@gmail.com	Hiba@9019	19/02/1990	Female	Bungalow B-12, PECHS Block 6	2022-08-01	Suspended	Orthodontist	D-103

1. **FIRST NORMAL FORM:**

   The table has an atomic value for every row and column intersection.
   Identifying Candidate Keys:

- (staff_id)
- (email)

We are selecting (staff_id) to be the Primary Key
1NF staff relation is described as follows:
Staff(<u>staff_id</u>, name_first, name_last, email, acc_password, dob, gender, address, date_joined, acc_status, staff_role, *dept_id*)

Identifying Anomalies:

- **Insertion Anomaly:**
  If a new department is created e.g. D-104, can be created independently in the **Department table** without requiring any staff assignment as dept_id exists as a standalone primary key in department table
- **Update Anomaly:**
  Any changes in department details (e.g., department name or room number for D-101) are updated **only in the Department table**.
- **Deletion Anomaly:**
  Deleting a staff record (e.g., Muhammad Bilal from D-102) does **not affect department information**.

Department data is stored independently in the Department table and remains intact even if all staff members are removed.

2. **SECOND NORMAL FORM:**

   FD1: **<u>staff_id</u>**, → name_first, name_last, email, acc_password, dob, gender, address, date_joined, acc_status, staff_role, *dept_id*
   As we do not have composite primary key, hence our table is already in 2NF

3. **THIRD NORMAL FORM:**

If we see our table,
staff_id → name_first, name_last, email, acc_password, dob, gender, address, date_joined, acc_status, staff_role, dept_id
We have transitive dependency i.e. staff_id → email → name_first, name_last, acc_password, dob, gender, address, date_joined, acc_status, staff_role, dept_id
As email can come under personal detail and is unique.
So, we separate the tables
Staff(**staff_id**, email)
Staff_Details(**email**, name_first, name_last, acc_password, dob, gender, address, date_joined, acc_status, staff_role, dept_id)

Staff:

staff_id	email
ST-000001	arsalan.ahmed88@gmail.com
ST-000002	sarah_munir92@hotmail.com
ST-000004	hiba.siddiqui90@gmail.com

Staff_Details

email	name_first	name_last	acc_password	dob	gender	address	date_joined	acc_status	staff_role	dept_id
arsalan.ahmed88@gmail.com	Dr. Arsalan	Ahmed	Arsalan@8812	12/04/1988	Male	House 42, Block L, North Nazimabad	2023-01-15	Active	Senior Dentist	D-101
sarah_munir92@hotmail.com	Sarah	Munir	Sarah@9205	25/08/1992	Female	Flat 302, Al-Azam Square, Gulshan	2024-05-10	Active	Dental Assistant	D-101
m.bilal.khi@outlook.com	Muhammad	Bilal	Bilal@9511	05/11/1995	Male	Sector 11-G, New Karachi	2025-02-20	Active	Receptionist	D-102
hiba.siddiqui90@gmail.com	Dr. Hiba	Siddiqui	Hiba@9019	19/02/1990	Female	Bungalow B-12, PECHS Block 6	2022-08-01	Suspended	Orthodontist	D-103

Patient_Record

record_id	patient_id	Staff_id	visit_date	procedure_done	next_appointment	total_amount	amount_paid	clinical_notes
1	PT-000001	ST-000001	2025-12-05	Consultation	2025-12-12	500	500	Initial checkup
2	PT-000001	ST-000001	2025-12-08	1/3 chamber opened	2025-12-15	100000	10000	RCT + Crown plan started
20	PT-000002	ST-000002	2026-02-10	Impression for U/A retainer taken	NULL	5000	5000	Initial impression
21	PT-000002	ST-000002	2026-02-14	Retainer Inserted	2026-02-21	0	0	Final fitting - NIL balance
25	PT-000003	ST-000001	2026-01-31	Chamber opened 4/6	2026-02-07	10000	5000	3 canals located; MB slightly constricted
26	PT-000003	ST-000002	2026-01-31	Cotton cavet placed	2026-02-03	1000	1000	Interim dressing

1. **FIRST NORMAL FORM:**

   The table has one atomic value at every level, (we are taking clinical notes a whole single sentence value attribute)
   Hence table is in 1NF
   Identifying Candidate Keys:

- (record_id)

  (patient_id, staff_id, visit_date) could have been a CK only if a patient can have a single record for a single day but it's not true for our database
  We choose record_id as Primary Key.

2. **SECOND NORMAL FORM:**

   We have only one PK so there is no partial dependency.
   Identifying Anomalies:

- **Insertion Anomaly:**

  A new patient treatment record can be inserted independently without needing to repeat patient or staff details, because patient_id and staff_id act as foreign keys referring to separate Patient and Staff entities.
- **Update Anomaly:**

  Any changes in staff or patients are updated **only in the Staff and Patient tables**.
- **Deletion Anomaly:**

  If we delete a record from the table, it won't delete staff and Patient details.
  Deleting a treatment record removes only that specific transaction's information (including payment details associated with that visit).

3. **THIRD NORMAL FORM:**

   There is no transitive dependency hence our table is already in 3NF.

Record_Clinical_Notes

record_id	clinical_notes
1	Initial checkup
2	RCT + Crown plan started
20	Initial impression

1. **FIRST NORMAL FORM:**

   The table is already 1NF as it has atomic value at every row column intersection.
   Identify candidate key/s:
   - (record_id, clinical_notes)

   We consider (record_id, clinical_notes) as the Primary Key
2. **SECOND NORMAL FORM:**

FD: (record_id, clinical_notes)
Table is already in 2NF as we have no non-PK attribute.
Identifying Anomalies:

- **Insertion Anomaly:**
  There is no insertion anomaly as each id is being unique identified.
- **Deletion Anomaly:**
  If we delete a record, we only lose the respective data as record is already present in Patient_Record table.
- **Update Anomaly:**
  Each row is unique hence, we don't have update anomaly.

3. **THIRD NORMAL FORM:**

FD: record_id, clinical_notes
There is no transitive dependency as we have just no non-PK attribute hence our table is already in 3NF.

Medical

staff_id	license_number	years_experience
ST-000001	LIC-2023-8891	12
ST-000002	LIC-2024-4520	3
ST-000004	LIC-2020-1105	6

1. **FIRST NORMAL FORM:**
   Table is already in 1NF as there is one atomic value at every row and column intersection.
   Identifying Candidate Keys:
- (staff_id)
- (license_number)

   We are selecting staff_id as the primary key
2. **SECOND NORMAL FORM:**
   FD1: staff_id → license_number, years_experience
   This is the only full dependency and there is no partial dependency. Hence table is already in 2NF.

   Identifying Anomalies:
- **Insertion Anomaly:**
  If we add license number, then we need to assign a dummy staff_id but this is not possible so there is an insertion Anomaly.
- **Deletion Anomaly:**
  If we delete a staff_id from it, it won't affect the original staff record in the original table but we may loose license number.
- **Update Anomaly:**
  There is no update anomaly in the table as each value is unique.

3. **THIRD NORMAL FORM:**
   FD1: staff_id → license_number, years_experience
   Here, staff_id → license_number and years_experience can be determined by checking their details through the license_number hence
   license_number → years_experience
   So there is a transitive dependency,
   Staff_id → license_number → years_experience
   To remove it we will separate tables

   Medical(**<u>staff_id</u>**, license_number)
   Medical_Details(**<u>license_number</u>**, years_experience)

Medical

staff_id	license_number
ST-000001	LIC-2023-8891
ST-000002	LIC-2024-4520
ST-000003	LIC-2025-7788
ST-000004	LIC-2020-1105

Medical_Details

license_number	years_experience
LIC-2023-8891	12
LIC-2024-4520	3

LIC-2025-7788	2
LIC-2020-1105	6

This removes our Insertion and Deletion Anomaly occurring.

Doctor

staff_id	availability
ST-000001	Available
ST-000002	Available
ST-000004	Available

1. **FIRST NORMAL FORM:**
   The table is already 1NF as it has atomic value at every row column intersection.
   Identify candidate key/s:
- (staff_id)

   We consider staff_id as the Primary Key
2. **SECOND NORMAL FORM:**
   FD1: staff_id → availability
   Table is already in 2NF as we have single attribute in PK.
   Identifying Anomalies:
- **Insertion Anomaly:**
  There is no insertion anomaly as each id is being unique identified.
- **Deletion Anomaly:**
  It won't lead to direct loss of data as we have already stored staff_id in Staff table
- **Update Anomaly:**
  Each row is unique hence, we don't have update anomaly.

3. **THIRD NORMAL FORM:**
   FD1: staff_id → availability
   There is no transitive dependency as only one non-PK attribute depends on the PK.

Department

dept_id	dept_name	room_number
D-101	Endodontics	10
D-103	Orthodontics	20

1. **FIRST NORMAL FORM:**
   The table is already 1NF as it has atomic value at every row column intersection.
   Identify candidate key/s:
- (dept_id)

   We consider dept_id as the Primary Key

2. **SECOND NORMAL FORM:**
   FD1: dept_id → dept_name

FD2: dept_id → room_number
Table is already in 2NF as we have single attribute in PK.

Identifying Anomalies:

- **Insertion Anomaly:**
  There is no insertion anomaly as each id is being unique identified.
- **Deletion Anomaly:**
  It won't lead to direct loss of data as we have already stored dept_id in Department table
- **Update Anomaly:**
  Each row is unique hence, we don't have update anomaly.

3. THIRD NORMAL FORM:
FD1: dept_id a dept_name
FD2: dept_id a room_number
There is no transitive dependency as only non-PK attributes depending on the PK.

Supplier

supplier_id	contact_person	company_name	email	phone	address
SUP-000001	Zafar Iqbal	Indus Dental Supplies	info@indusdental.com	021-34567890	Plot 24, Korangi Industrial Area, Karachi
SUP-000002	Maria Abbas	MedLine Pakistan	sales@medline.pk	021-38251442	Suite 102, Business Plaza, Shahra-e-Faisal
SUP-000003	Kamran Shah	Karachi Ortho Tools	support@orthotools.com	021-36641234	Shop 5, Al-Najeebi Market, Saddar
SUP-000004	Dr. Naveed	Precision Lab Gear	sales@precisiongear.com	021-34990081	Block 13-C, Gulshan-e-Iqbal, Karachi

1. **FIRST NORMAL FORM:**
   The table is already 1NF as it has atomic value at every row column intersection.
   Identify candidate key/s:

- (supplier_id)
- (email)
- (phone)

We consider supplier_id as the Primary Key

2. **SECOND NORMAL FORM:**
   FD1: supplier_id → contact_person, company_name, email, phone, address
   There is no partial dependency as each attribute fully depends on the PK.

3. **THIRD NORMAL FORM:**
   FD1: supplier_id → contact_person, company_name, email, phone, address
   If we see the table,
   Supplier_id → email → company_name
   Hence email transitively depends on supplier_id

We can make a new table as supplier_details

Supplier(<u>supplier_id</u>, contact_person, email)
Supplier_Details(<u>email</u>, company_name, phone, address)

Supplier

supplier_id	contact_person	email
SUP-000001	Zafar Iqbal	info@indusdental.com
SUP-000002	Maria Abbas	sales@medline.pk
SUP-000003	Kamran Shah	support@orthotools.com
SUP-000004	Dr. Naveed	sales@precisiongear.com

Supplier_Details

email	company_name	phone	address
info@indusdental.com	Indus Dental Supplies	021-34567890	Plot 24, Korangi Industrial Area, Karachi
sales@medline.pk	MedLine Pakistan	021-38251442	Suite 102, Business Plaza, Shahra-e-Faisal
support@orthotools.com	Karachi Ortho Tools	021-36641234	Shop 5, Al-Najeebi Market, Saddar
sales@precisiongear.com	Precision Lab Gear	021-34990081	Block 13-C, Gulshan-e-Iqbal, Karachi

Bill

bill_id	amount_paid	Payment_date	record_id	payment_method
B-000000001	500	2025-12-05	1	Cash
B-000000002	10000	2025-12-08	2	Bank Transfer
B-000000003	10000	2025-02-08	2	Cash
B-000000004	10000	2026-02-31	2	Cash
B-000000005	10000	2026-02-31	2	Easypaisa
B-000000004	10000	2026-02-31	2	Cash

1. FIRST NORMAL FORM:
The table is already 1NF as it has atomic value at every row column intersection.
Identify candidate key/s:

- (bill_id)

We consider bill_id as the Primary Key

2. SECOND NORMAL FORM:
FD: bill_id → amount_paid, payment_date, payment_method, *record_id*
Table is already in 2NF as we have single attribute in PK.

Identifying Anomalies:

- **Insertion Anomaly:**
  There is no insertion anomaly as each id is being unique identified.
- **Deletion Anomaly:**
  If we delete a bill, we only lose that bill's data.
- **Update Anomaly:**
  Each installment is a unique event; hence, we don't have update anomaly.

3. THIRD NORMAL FORM:
FD: bill_id → amount_paid, payment_date, payment_method, *record_id*
There is no transitive dependency hence our table is already in 3NF.

Administrative

staff_id	clearance_level	budget_authority
ST-000005	Level-3	150000
ST-000006	Level-2	50000
ST-000007	Level-2	30000
ST-000008	Level-3	200000

1. FIRST NORMAL FORM:
The table is already 1NF as it has atomic value at every row column intersection.
Identify candidate key/s:

- (staff_id)

We consider staff_id as the Primary Key

2. SECOND NORMAL FORM:
FD: staff_id → clearance_level, budget_authority
Table is already in 2NF as we have single attribute in PK.

Identifying Anomalies:

- **Insertion Anomaly:**
  There is no insertion anomaly as each id is being unique identified.
- **Deletion Anomaly:**
  If we delete a staff, we only lose the respective data as staff is already present in Staff table.
- **Update Anomaly:**
  Each row is unique hence, we don't have update anomaly.

3. THIRD NORMAL FORM:
FD: staff_id → clearance_level, budget_authority

There is no transitive dependency hence our table is already in 3NF.

Nurse

staff_id	shift_type
ST-000009	Morning
ST-000010	Night

1. FIRST NORMAL FORM:
The table is already 1NF as it has atomic value at every row column intersection.
Identify candidate key/s:

- (staff_id)

We consider staff_id as the Primary Key

2. SECOND NORMAL FORM:
FD: staff_id → shift_type
Table is already in 2NF as we have single attribute in PK.

Identifying Anomalies:

- **Insertion Anomaly:**
  There is no insertion anomaly as each id is being unique identified.
- **Deletion Anomaly:**
  If we delete a staff, we only lose the respective data as staff is already present in Staff table.
- **Update Anomaly:**
  Each row is unique hence, we don't have update anomaly.

3. THIRD NORMAL FORM:
FD: staff_id → shift_type
There is no transitive dependency as we have just one non-PK attribute hence our table is already in 3NF.

Staff_Phone

staff_id	phone
ST-000001	0321-2456789
ST-000001	0300-8214356
ST-000002	0321-3834912
ST-000003	0321-3445932
ST-000004	0331-1272554

4. **FIRST NORMAL FORM:**

   The table is already 1NF as it has atomic value at every row column intersection.
   Identify candidate key/s:
   - (staff_id, phone)

   We consider (staff_id, phone) as the Primary Key
5. **SECOND NORMAL FORM:**

FD: (staff_id, phone)
Table is already in 2NF as we have no non-PK attribute.
Identifying Anomalies:
- **Insertion Anomaly:**
  There is no insertion anomaly as each id is being unique identified.
- **Deletion Anomaly:**
  If we delete a staff, we only lose the respective data as staff is already present in Staff table.
- **Update Anomaly:**
  Each row is unique hence, we don't have update anomaly.

6. **THIRD NORMAL FORM:**

FD: (staff_id, phone)
There is no transitive dependency as we have just no non-PK attribute hence our table is already in 3NF.

Patient_Phone

patient_id	phone
PT-000001	03492585721
PT-000002	0321-3834149
PT-000002	0315-9988776
PT-000003	0331-1272396
PT-000004	0300-4455667

1. **FIRST NORMAL FORM:**

   The table is already 1NF as it has atomic value at every row column intersection.
   Identify candidate key/s:
- (patient_id, phone)

   We consider (patient_id, phone) as the Primary Key
2. **SECOND NORMAL FORM:**

   FD: (patient_id, phone)
   Table is already in 2NF as we have no non-PK attribute.
   Identifying Anomalies:
- **Insertion Anomaly:**
  There is no insertion anomaly as each id is being unique identified.

- **Deletion Anomaly:**
  If we delete a patient, we only lose the respective data as patient is already present in Patients table.
- **Update Anomaly:**
  Each row is unique hence, we don't have update anomaly.

3. **THIRD NORMAL FORM:**

   FD: (patient_id, phone)
   There is no transitive dependency as we have no non-PK attribute hence our table is already in 3NF.

Doctor_Specialization

staff_id	specialization
ST-000001	Endodontist
ST-000001	Endodontic Surgery
ST-000002	Orthodontist
ST-000004	Orthodontist
ST-000004	Dentofacial Orthopedics

1. **FIRST NORMAL FORM:**

   The table is already 1NF as it has atomic value at every row column intersection.
   Identify candidate key/s:
- (staff_id, specialization)

  We consider (staff_id, specialization) as the Primary Key

2. **SECOND NORMAL FORM:**

   FD: (staff_id, specialization)
   Table is already in 2NF as we have no non-PK attribute.
   Identifying Anomalies:
- **Insertion Anomaly:**
  There is no insertion anomaly as each id is being unique identified.
- **Deletion Anomaly:**
  If we delete a staff, we only lose the respective data as staff is already present in Staff table.
- **Update Anomaly:**
  Each row is unique hence, we don't have update anomaly.

3. **THIRD NORMAL FORM:**

   FD: (staff_id, specialization)
   There is no transitive dependency as we have no one non-PK attribute hence our table is already in 3NF.

Doctor_Qualification

staff_id	qualification
ST-000001	BDS
ST-000001	FCPS
ST-000002	BDS
ST-000002	MCPS
ST-000004	FCPS (Ortho)

1. **FIRST NORMAL FORM:**

   The table is already 1NF as it has atomic value at every row column intersection.
   Identify candidate key/s:
   - (staff_id, qualification)

   We consider (staff_id, qualification) as the Primary Key
2. **SECOND NORMAL FORM:**

   FD: (staff_id, qualification)
   Table is already in 2NF as we have no non-PK attribute.
   Identifying Anomalies:
   - **Insertion Anomaly:**
     There is no insertion anomaly as each id is being unique identified.
   - **Deletion Anomaly:**
     If we delete a staff, we only lose the respective data as staff is already present in Staff table.
   - **Update Anomaly:**
     Each row is unique hence, we don't have update anomaly.

3. **THIRD NORMAL FORM:**

   FD: (staff_id, qualification)
   There is no transitive dependency as we have no non-PK attribute hence our table is already in 3NF.

Salary

staff_id (PK)	payment_date (PK)	amount	bonus	deduction
ST-000001	2026-01-31	150000	5000	2000
ST-000002	2026-01-31	45000	2000	500
ST-000003	2026-01-31	85000	3000	1000
ST-000001	2026-02-28	150000	0	3000
ST-000004	2026-02-28	120000	0	5000

1. **FIRST NORMAL FORM:**

The table has an atomic value for every row and column intersection.
Identifying Candidate Keys:

- (staff_id, payment_date)

We are selecting (staff_id, payment_date) to be the Composite Primary Key.
1NF salary relation is described as follows:
Salary (**staff_id**, **payment_date**, amount, bonus, deduction, staff_name, staff_role).

Identifying Anomalies:

- **Insertion Anomaly:**
  There is no insertion anomaly as each id is being uniquely identified.
- **Deletion Anomaly:**
  If we delete a staff, we only lose the respective data as staff is already present in Staff table.
- **Update Anomaly:**
  Each row is unique as salary needs not to be updated in past. Hence, we don't have update anomaly.

2. **SECOND NORMAL FORM:**

FD: staff_id, payment_date → amount, bonus, deduction
All non-PK values depend directly on (staff_id, payment_date). If we consider either staff_id or payment_date then we can have multiple values of amount, bonus, and deduction. Hence they both cannot identify any non-PK value separately.
The table is already in 2NF.

3. **THIRD NORMAL FORM:**

If we see our tables:
staff_id, payment_date→ amount, bonus, deduction.
In the table, all non-key attributes (amount, bonus, deduction) depend directly and only on the composite primary key. There are no transitive dependencies in the table.

Products

item_id (PK)	item_name	category	quantity	unit	unit_price	supply_date	quantity_supplied	supplier_id
I-0001	Composite Resin	Restorative	50	Tube	2500	2026-03-10	50	S-00001
I-0001	Composite Resin	Restorative	20	Tube	2500	2026-04-05	20	S-00001
I-0002	Dental Mirror	Diagnostic	15	Piece	450	2026-04-10	15	S-00002
I-0004	Alginate	Impression	10	Packet	1800	2026-04-12	10	S-00001

1. **FIRST NORMAL FORM:**

The table is in **1NF** because all values are atomic and there are no repeating groups. **Identify candidate key/s:**

- (item_id, supply_date, supplier_id)

Because an item can be supplied by the same supplier on different dates, we must use this **composite primary key** to uniquely identify each row.

2. **SECOND NORMAL FORM:**

Functional Dependencies (FDs):

- {item_id, supply_date, supplier_id} → quantity_supplied
- **item_id** → item_name, category, unit, unit_price, quantity **(Partial Dependency)**

The item's name and price depend only on the item_id, not the whole composite key.

Identifying Anomalies:

- **Insertion Anomaly:** You cannot add a new product to your system without having a specific supply date and a supplier.
- **Deletion Anomaly:** If you delete a supply record to clear history, you accidentally delete the product's name and price from your system.
- **Update Anomaly:** If the price of "Composite Resin" changes, you must update every single row where that item has been supplied, or the data becomes inconsistent.

We split the table to remove partial dependencies.

- **Product** (**item_id**, item_name, category, unit, unit_price, quantity)
- **Product_Supplied** (**item_id**, **supplier_id**, supply_date, quantity-supplied)

Product_Supplied

item_id	supplier_id	supply_date	quantity_supplied
I-0001	S-00001	2026-03-10	50
I-0001	S-00001	2026-04-05	20
I-0002	S-00002	2026-04-10	15
I-0004	S-00001	2026-04-12	10

Product

item_id	item_name	category	quantity	unit	unit_price
I-0001	Composite Resin	Restorative	70	Tube	2500
I-0002	Dental Mirror	Diagnostic	15	Piece	450
I-0003	Lidocaine 2%	Anesthetic	100	Vial	120
I-0004	Alginate	Impression	10	Packet	1800

3. **THIRD NORMAL FORM:**

- item_id → item_name, category, unit, unit_price, quantity
- (<u>item_id</u>, supplier_id, supply_date) → quantity_supplied

In the **Product** table, all non-key attributes (name, category, price) depend directly on the item_id.
In the **Product_Supplied** table, the quantity depends directly on the combination of the item, supplier, and date.
Hence there is no transitive dependency.

Patients

patient_id	name_first	name_last	email	acc_password	dob	gender	address	marital_status	diabetes	hypertension	allergies
PT-000001	Nasir	Ismail	nasir@gmail.com	Nasir@0349	15/05/1985	Male	Kaltor House	Married	TRUE	TRUE	FALSE
PT-000002	Rabiha Khulood	G. Hussain	rabiha@gmail.com	Rabiha@0321	22/08/1992	Female	Nazimabad No. 4	Married	FALSE	FALSE	FALSE
PT-000003	Ali Haider	Roshan Ali	ali@gmail.com	Ali@0331	10/11/1998	Male	Baldia Town	Single	FALSE	FALSE	FALSE
PT-000004	Kehkashan	Suleman	suleman@gmail.com	Suleman@123	05/02/1975	Female	Nazimabad No. 4	Married	FALSE	TRUE	FALSE

1. FIRST NORMAL FORM (1NF)
The table has an atomic value for every row and column intersection.
Identify Candidate Key/s:

- (patient_id)
- (Email)

We consider **patient_id** as the Primary Key. The 1NF table can be given as:
Patient(**<u>patient_id</u>**, name_first, name_last, email, chief_complaint, acc_password, dob, gender, address, marital_status, diabetes, hypertension, allergies)
2. SECOND NORMAL FORM (2NF)
FD: patient_id → name_first, name_last, email, chief_complaint, acc_password, dob, gender, address, marital_status, diabetes, hypertension, allergies
Table is already in **2NF** as the Primary Key consists of a single attribute (patient_id), meaning no partial functional dependencies can exist.
Identifying Anomalies:

- **Insertion Anomaly:** None; a new patient can be registered with a unique ID even if they don't have a treatment record yet.
- **Update Anomaly:** If a patient's contact information changes, we only need to update it in this single row.
- **Deletion Anomaly:** Deleting a treatment record in another table does not result in the loss of the patient's core demographic or medical status (diabetes/hypertension) because they are stored here.

3. THIRD NORMAL FORM (3NF)
FD1: patient_id → email
FD2: email → name_first, name_last, chief_complaint, acc_password, dob, gender, address, marital_status, diabetes, hypertension, allergies
There is a **Transitive Dependency**. Since Email is unique and can identify the person, the relationship follows the pattern: patient_id → email → name_first, name_last, gender, dob, address, phone.
To achieve **3NF**, we decompose the relation into a Patient table and a Details table.

- **Patient** (patient_id, email)
- **Patient_Details** (*email*, name_first, name_last, email, chief_complaint, acc_password, dob, gender, address, marital_status, diabetes, hypertension, allergies)

Patient

patient_id	email
PT-000001	nasir@gmail.com
PT-000002	rabiha@gmail.com
PT-000003	ali@gmail.com
PT-000004	suleman@gmail.com

Patient_Details

email	acc_password	name_first	name_last	dob	gender	address	marital_status	diabetes	hypertension	allergies
nasir@gmail.com	Nasir@0349	Nasir	Ismail	15/05/1985	Male	Kaltor House	Married	TRUE	TRUE	FALSE
rabiha@gmail.com	Rabiha@0321	Rabiha Khulood	G. Hussain	22/08/1992	Female	Nazimabad No. 4	Married	FALSE	FALSE	FALSE
ali@gmail.com	Ali@0331	Ali Haider	Roshan Ali	10/11/1998	Male	Baldia Town	Single	FALSE	FALSE	FALSE
suleman@gmail.com	Suleman@123	Kehkashan	Suleman	05/02/1975	Female	Nazimabad No. 4	Married	FALSE	TRUE	FALSE

Procurement

staff_id	supplier_id
ST-000003	S-00001
ST-000003	S-00002

ST-000004	S-00001

1. **FIRST NORMAL FORM:**

The table is already in **1NF** as it has atomic values at every row-column intersection.
Identify candidate key/s:

- (staff_id, supplier_id)

We consider the composite key **(staff_id, supplier_id)** as the Primary Key

2. **SECOND NORMAL FORM:**

There are no non-key attributes in this table. Table is already in **2NF** because there are no partial functional dependencies of non-key attributes on the primary key.

3. **THIRD NORMAL FORM:**

Since there are no non-key attributes at all, there are no transitive dependencies. Hence, the table is already in **3NF**.

Manages

item_id	staff_id
I-0001	ST-000003
I-0002	ST-000003
I-0003	ST-000004

1. **FIRST NORMAL FORM (1NF):**

The table is already in **1NF** as it has atomic values at every row-column intersection. **Identify candidate key/s:**

- (item_id, staff_id)

We consider the composite key **(item_id, staff_id)** as the Primary Key.

2. **SECOND NORMAL FORM (2NF):**

Table is already in **2NF** as we have no non-key attributes that could partially depend on the primary key.

3. **THIRD NORMAL FORM (3NF):**

There is no transitive dependency as there are no non-key attributes; our table is already in **3NF**.

Appointment

staff_id	patient_id	appointment_date	appointment_time
ST-000001	PT-000001	2026-04-15	10:00 AM
ST-000001	PT-000002	2026-04-15	11:30 AM
ST-000004	PT-000001	2026-04-20	02:00 PM

1. **FIRST NORMAL FORM (1NF):**

The table is already in **1NF** as it has atomic values. **Identify candidate key/s:**

- (staff_id, patient_id, appointment_date)

We consider the composite key **(staff_id, patient_id, appointment_date)** as the Primary Key (assuming a patient only sees the same doctor once per day).

2. **SECOND NORMAL FORM (2NF):**

FD: (staff_id, patient_id, appointment_date) → appointment_time
Table is in **2NF** as the non-key attribute (appointment_time) depends on the **full** composite primary key.
Identifying Anomalies:

- **Insertion Anomaly:**
  There is no insertion anomaly as each id is being unique identified.
- **Deletion Anomaly:**
  If we delete a row, we only lose the respective data as staff and patient are already present in Staff and Patient table respectively.
- **Update Anomaly:**
  There can be an update anomaly when a doctor has to change his slot, it can result in all slots being updated. But we consider doctors can only update there time if it's available else no.

3. **THIRD NORMAL FORM (3NF):**

FD: (staff_id, patient_id, appointment_date) → appointment_time
There is no transitive dependency between non-key attributes (as there is only one non-key attribute); hence our table is already in **3NF**.

Final Normalized Tables

For Strong Entity Sets

- **Department(<u>dept_id</u>**, dept_name, room_number)

- **Patient** (**<u>patient_id</u>**, *email*)
- **Patient_Details** (**<u>email</u>**, name_first, name_last, email, chief_complaint, acc_password, dob, gender, address, marital_status, diabetes, hypertension, allergies)

- **Supplier(<u>supplier_id</u>**, contact_person, *email*)
- **Supplier_Details(<u>email</u>**, company_name, phone, address)

- **Staff(<u>staff_id</u>**, email)
- **Staff_Details(<u>email</u>**, name_first, name_last, acc_password, dob, gender, address, date_joined, acc_status, staff_role, *dept_id*)

- **Product** (**item_id**, item_name, category, unit, unit_price, quantity)
- **Product_Supplied** (**<u>item_id</u>**, **<u>supplier_id,</u>** supply_date, quantity-supplied)

- **Patient_Record(<u>record_id</u>**, treatment_plan, prescription, procedure_done, visit_date, next_appointment, total_amount, amount_paid, *patient_id, staff_id*)

- **Bill(<u>bill_id</u>**, amount_paid, payment_date, payment_method, *record_id*)

For Schema Derived From Isa Relationships

- **Medical(<u>staff_id</u>**, *license_number*)
- **Medical_Details(<u>license_number</u>**, years_experience)

- **Doctor(<u>staff_id</u>**, availability)
- **Administrative(<u>staff_id</u>**, clearance_level, budget_authority)
- **Nurse(<u>staff_id,</u>** shift_type)

For Multivalued Attributes Relational Schema

- **Staff_Phone(*<u>staff_id</u>*, <u>phone</u>**)
- **Patient_Phone(*<u>patient_id</u>*, <u>phone</u>**)
- **Doctor_Specialization(*<u>staff_id</u>*, <u>specialization</u>**)
- **Doctor_ Qualification** (***<u>staff_id</u>*, <u>qualification</u>**)

- **Record_Clinical_Notes**(***record_id***, **clinical_notes**)

For Weak Entity Sets

- **Salary**(***staff_id***, **payment_date**, amount, bonus, deduction)

For Schemas Derived From A Relationship Set Involving Strong Entity Sets

- **Procurement**(**staff_id**, supplier_id)
- **Manages**(**item_id**, **staff_id**)
- **Appointment**(**staff_id**, **patient_id,** appointment_date, appointment_time)

Note: The Primary Keys of each relation are indicated by bold and underlines, while the Foreign Keys are represented by *italics*.

DDL Statements For Creating Tables

Strong Entity Sets

```
-- Table: Department
CREATE TABLE Department (
   dept_id VARCHAR(10) PRIMARY KEY,
   dept_name VARCHAR(100) NOT NULL,
   room_number VARCHAR(10)
);

-- Table: Supplier_Details (Parent for Supplier)
CREATE TABLE Supplier_Details (
   email VARCHAR(100) PRIMARY KEY,
   company_name VARCHAR(100) NOT NULL,
   phone VARCHAR(20),
   address TEXT
);

-- Table: Supplier
CREATE TABLE Supplier (
   supplier_id VARCHAR(10) PRIMARY KEY,
   contact_person VARCHAR(100),
   email VARCHAR(100),
   FOREIGN KEY (email) REFERENCES Supplier_Details(email)
);

-- Table: Staff_Details (Parent for Staff)
CREATE TABLE Staff_Details (
   email VARCHAR(100) PRIMARY KEY,
   name_first VARCHAR(50) NOT NULL,
   name_last VARCHAR(50) NOT NULL,
   acc_password VARCHAR(255) NOT NULL,
   dob DATE,
   gender VARCHAR(10),
   address TEXT,
   date_joined DATE,
   acc_status VARCHAR(20),
   staff_role VARCHAR(50),
   dept_id VARCHAR(10),
   FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
```

```
-- Table: Staff
CREATE TABLE Staff (
    staff_id VARCHAR(10) PRIMARY KEY,
    email VARCHAR(100),
    FOREIGN KEY (email) REFERENCES Staff_Details(email)
);
-- Table: Patient_Details
CREATE TABLE Patient_Details (
    email VARCHAR(100) PRIMARY KEY,
    name_first VARCHAR(50),
    name_last VARCHAR(50),
    gender VARCHAR(10),
    marital_status VARCHAR(20),
    phone VARCHAR(20),
    address TEXT,
    has_diabetes BOOLEAN,
    has_hypertension BOOLEAN,
    has_allergies BOOLEAN,
    dob DATE
);
-- Table: Patient
CREATE TABLE Patient (
    patient_id VARCHAR(10) PRIMARY KEY,
    email VARCHAR(100) REFERENCES Patient_Details(email),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Table: Patient_Record
CREATE TABLE Patient_Record (
    record_id VARCHAR(10) PRIMARY KEY,
    treatment_plan TEXT,
    prescription TEXT,
    procedure_done VARCHAR(255),
    visit_date DATE,
    next_appointment DATE,
    total_amount DECIMAL(12, 2),
    amount_paid DECIMAL(12, 2),
    patient_id VARCHAR(10), -- Foreign Key to Patients table
    staff_id VARCHAR(10),   -- Doctor/Staff who treated
    FOREIGN KEY (staff_id) REFERENCES Staff(staff_id)
);
```

```
-- Table: Bill
CREATE TABLE Bill (
   bill_id SERIAL PRIMARY KEY,
   amount_paid DECIMAL(12, 2),
   payment_date DATE,
   payment_method VARCHAR(50),
   record_id INT,
   FOREIGN KEY (record_id) REFERENCES Patient_Record(record_id)
);
```

ISA Relationships (Specialization)

```
-- Table: Medical_Details
CREATE TABLE Medical_Details (
   license_number VARCHAR(50) PRIMARY KEY,
   years_experience INT
);

-- Table: Medical (Sub-type of Staff)
CREATE TABLE Medical (
   staff_id VARCHAR(10) PRIMARY KEY,
   license_number VARCHAR(50),
   FOREIGN KEY (staff_id) REFERENCES Staff(staff_id),
   FOREIGN KEY (license_number) REFERENCES Medical_Details(license_number)
);

-- Table: Doctor (Sub-type of Medical)
CREATE TABLE Doctor (
   staff_id VARCHAR(10) PRIMARY KEY,
   availability TEXT,
   FOREIGN KEY (staff_id) REFERENCES Staff(staff_id)
);

-- Table: Administrative (Sub-type of Staff)
CREATE TABLE Administrative (
   staff_id VARCHAR(10) PRIMARY KEY,
   clearance_level VARCHAR(20),
   budget_authority DECIMAL(15, 2),
   FOREIGN KEY (staff_id) REFERENCES Staff(staff_id)
);

-- Table: Nurse (Sub-type of Medical Staff)
CREATE TABLE Nurse (
        staff_id VARCHAR(10) PRIMARY KEY,
        shift_type VARCHAR(20) NOT NULL,
        FOREIGN KEY (staff_id) REFERENCES Staff(staff_id) );
```

3. Multivalued Attributes

-- Table: Staff_Phone

```
CREATE TABLE Staff_Phone (
   staff_id VARCHAR(10) NOT NULL,
   phone VARCHAR(20) NOT NULL,
   PRIMARY KEY (staff_id, phone),
   FOREIGN KEY (staff_id) REFERENCES Staff(staff_id)
);
```

-- Table: Patient_Phone

```
CREATE TABLE Patient_Phone (
   patient_id VARCHAR(10) NOT NULL,
   phone VARCHAR(20),
   PRIMARY KEY (patient_id, phone)
   -- Assuming a Patient table exists as the parent
);
```

-- Table: Doctor_Specialization

```
CREATE TABLE Doctor_Specialization (
   staff_id VARCHAR(10) NOT NULL,
   specialization VARCHAR(100),
   PRIMARY KEY (staff_id, specialization),
   FOREIGN KEY (staff_id) REFERENCES Doctor(staff_id)
);
```

-- Table: Doctor_Qualification

```
CREATE TABLE Doctor_Qualification (
   staff_id VARCHAR(10) NOT NULL,
   qualification VARCHAR(100),
   PRIMARY KEY (staff_id, qualification),
   FOREIGN KEY (staff_id) REFERENCES Doctor(staff_id)
);
```

-- Table: Record_Clinical_Notes

```
CREATE TABLE Record_Clinical_Notes (
   record_id VARCHAR(10) NOT NULL,
   clinical_notes TEXT,
   PRIMARY KEY (record_id, clinical_notes),
   FOREIGN KEY (record_id) REFERENCES Patient_Record(record_id)
);
```

Weak Entity Sets

-- Table: Salary (Weak entity depending on Staff)

```
CREATE TABLE Salary (
```

```
    staff_id VARCHAR(10) NOT NULL,
    payment_date DATE,
    amount DECIMAL(12, 2) NOT NULL,
    bonus DECIMAL(12, 2) DEFAULT 0,
    deduction DECIMAL(12, 2) DEFAULT 0,
    PRIMARY KEY (staff_id, payment_date),
    FOREIGN KEY (staff_id) REFERENCES Staff(staff_id)
);
```

Schemas Derived From A Relationship Set Involving Strong Entity Sets

l -- Table: Procurement

```
CREATE TABLE Procurement (
    staff_id VARCHAR(10) NOT NULL,
    supplier_id VARCHAR(10),
    PRIMARY KEY (staff_id, supplier_id),
    FOREIGN KEY (staff_id) REFERENCES Administrative(staff_id),
    FOREIGN KEY (supplier_id) REFERENCES Supplier(supplier_id)
);
```

-- Table: Manages

```
CREATE TABLE Manages (
    item_id VARCHAR(10) NOT NULL,
    staff_id VARCHAR(10) NOT NULL,
    PRIMARY KEY (item_id, staff_id),
    FOREIGN KEY (item_id) REFERENCES Product(item_id),
    FOREIGN KEY (staff_id) REFERENCES Administrative(staff_id)
);
```

-- Table: Appointment

```
CREATE TABLE Appointment (
    staff_id VARCHAR(10) NOT NULL,
    patient_id VARCHAR(10) NOT NULL,
    appointment_date DATE,
    appointment_time TIME,
    PRIMARY KEY (staff_id, patient_id, appointment_date),
    FOREIGN KEY (staff_id) REFERENCES Staff(staff_id),
    FOREIGN KEY (patient_id) REFERENCES Patient(patient_id)
);
```

DDL Statements For Indices:

-- Patient Table Indices

```
CREATE INDEX idx_PatientDetails_FName ON Patient_Details (name_first);
CREATE INDEX idx_PatientDetails_LName ON Patient_Details (name_last);
CREATE INDEX idx_PatientDetails_Phone ON Patient_Details (phone);
```

-- Patient Phone Table Index
CREATE INDEX idx_PatientPhone_PID ON Patient_Phone (patient_id);

-- Staff Details Table Indices
CREATE INDEX idx_StaffDetails_Email ON Staff_Details (email);
CREATE INDEX idx_StaffDetails_Password ON Staff_Details (acc_password);
CREATE INDEX idx_StaffDetails_Role ON Staff_Details (staff_role);
CREATE INDEX idx_StaffDetails_DeptID ON Staff_Details (dept_id);

-- Staff Phone Table Index
CREATE INDEX idx_StaffPhone_SID ON Staff_Phone (staff_id);

-- Medical Table Index
CREATE INDEX idx_Medical_License ON Medical (license_number);

-- Doctor Specialization & Qualification Indices
CREATE INDEX idx_Doctor_Spec ON Doctor_Specialization (specialization);
CREATE INDEX idx_Doctor_Qual ON Doctor_Qualification (qualification);
CREATE INDEX idx_Doctor_Availability ON Doctor (availability);

-- Patient Record Table Indices
CREATE INDEX idx_Record_VisitDate ON Patient_Record (visit_date);
CREATE INDEX idx_Record_NextAppt ON Patient_Record (next_appointment);
CREATE INDEX idx_Record_PatientID ON Patient_Record (patient_id);
CREATE INDEX idx_Record_StaffID ON Patient_Record (staff_id);

-- Bill Table Indices
CREATE INDEX idx_Bill_PaymentDate ON Bill (payment_date);
CREATE INDEX idx_Bill_RecordID ON Bill (record_id);

-- Salary Table Indices
CREATE INDEX idx_Salary_PayDate ON Salary (payment_date);

-- Supplier Table Indices
CREATE INDEX idx_Supplier_Email ON Supplier (email);
CREATE INDEX idx_SupplierDetails_Company ON Supplier_Details (company_name);

Acknowledgement

Usman Rasheed Siddiqui (CS-24038)

Developed the Main ER model, designed the Relational Schema, performed Normalization and designed final Normalized tables.

Huzaifa Hanif (CS-24039)

Prepared the introduction, gathered system requirements, and did system Formulation and Analysis.

Salman Raza (CS-24043)

Did SQL DDL queries for table creation and indexing.